Lab1 Report: Deploying Gitea with Docker and Verifying Git LFS Functionality

Student Information

Student ID: 202483910036

Name: Zhou Jiaqi

1. Objective

The core objectives of this experiment are:

1. Deploy a self-hosted Git service Gitea using Docker on WSL (Windows Subsystem for Linux).
2. Verify that Gitea supports Git LFS (Large File Storage) for managing large files, including resolving common operation errors during the process.
3. Practice using Git command-line operations (no cloning) to push tracked files to the self-hosted Gitea repository.

2. Environment Preparation

2.1 System Environment

Host OS: Windows 11

Virtualization Layer: WSL 2 (Ubuntu distribution)

Container Engine: Docker Desktop (integrated with WSL 2)

Gitea Version: `gitea/gitea:latest` (Docker image)

Git Version: 2.34.1 (WSL environment)

2.2 Core Dependencies

Docker & WSL 2 installed and running properly.

Git LFS installed and initialized in the local repository.

Gitea repository created on the local `localhost:3000` server.

3. Experimental Steps & Problem Resolution

3.1 Step 1: Deploy Gitea via Docker (WSL Environment)

First, I clean up the old Gitea container (if any) and redeploy a new container with correct port mapping and data persistence configuration.

```
zgg@LAPTOP-L21QMK1J: ~
wsl: 检测到 localhost 代理配置，但未镜像到 WSL。NAT 模式下的 WSL 不支持 localhost 代理。
docker pszgg@LAPTOP-L21QMK1J:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
zgg@LAPTOP-L21QMK1J:~$ docker rm -f gitea
gitea
zgg@LAPTOP-L21QMK1J:~$ docker run -d \
  --name=gitea \
  -p 3000:3000 \
  -p 222:22 \
  -v gitea_data:/data \
  --restart=always \
  gitea/gitea:latest
0cb8ccd9b6c7018159c1b8d353ef21443682f26a4aa7b90505b5b4ed5d440220
zgg@LAPTOP-L21QMK1J:~$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS
                                                           NAMES
0cb8ccd9b6c7   gitea/gitea:latest   "/usr/bin/entrypoint…"   18 seconds ago   Up 17 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:222->22/tcp, [::]:222->22/tcp   gitea
```

3.2 Step 2: Initialize Local Git Repository and Configure Git LFS

Enter the local directory `D:\Gitea-LFS-Test` (WSL path: `/mnt/d/Gitea-LFS-Test`), initialize the Git repository, and configure Git LFS to track large file types (`.zip`, `.iso`, `.bin`).

```
zgg@LAPTOP-L21QMK1J:~$ cd /mnt/d/Gitea-LFS-Test
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /mnt/d/Gitea-LFS-Test/.git/
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git remote add origin http://localhost:3000/zjq/lfs-test-new.git
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git lfs install
Updated Git hooks.
Git LFS initialized.
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git lfs track "*.zip"
git lfs track "*.iso"
git lfs track "*.bin"
Tracking "*.zip"
Tracking "*.iso"
Tracking "*.bin"
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git add .gitattributes
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git commit -m "init lfs"
```

3.3 Step 3: Resolve Git Identity Configuration Error

During the first commit, Git reports an "Author identity unknown" error because the global user name/email is not configured. I resolve this by setting the user information.

```
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git commit -m "init lfs"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: empty ident name (for <zgg@LAPTOP-L21QMK1J.localdomain>) not allowed
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git config --global user.name "zjq"
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git config --global user.email "1794671582@qq.com"
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git commit -m "init lfs"
[master (root-commit) 036fbad] init lfs
 1 file changed, 3 insertions(+)
```

3.4 Step 4: Resolve "Untracked Files" and Branch Mismatch Issues

When executing `git add .` and `git commit`, the terminal prompts that `liora.zip` is an untracked file (LFS tracking not take full effect), and the local branch is `master` while the remote branch is `main` (Gitea's default branch). I resolve these by re-tracking the file and renaming the branch.

```
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git commit -m "upload 1GB file via LFS"
On branch master
nothing to commit, working tree clean
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git commit -m "upload 1GB file via LFS"
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        liora.zip

nothing added to commit but untracked files present (use "git add" to track)
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git lfs track "liora.zip"
Tracking "liora.zip"
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git add liora.zip
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git add .gitattributes
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git commit -m "upload liora.zip via LFS"
[master 2819526] upload liora.zip via LFS
 2 files changed, 4 insertions(+)
 create mode 100644 liora.zip
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git branch -M main
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git push -u origin main
Username for 'http://localhost:3000': zjq
Password for 'http://zjq@localhost:3000':
```

```
Locking support detected on remote "origin". Consider enabling it with:
  $ git config lfs.http://localhost:3000/zjq/lfs-test-new.git/info/lfs.locksverify true
Uploading LFS objects: 100% (1/1), 61 KB | 0 B/s, done.
To http://localhost:3000/zjq/lfs-test-new.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'http://localhost:3000/zjq/lfs-test-new.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git pull origin main --allow-unrelated-histories
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 205 bytes | 2.00 KiB/s, done.
From http://localhost:3000/zjq/lfs-test-new
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
```

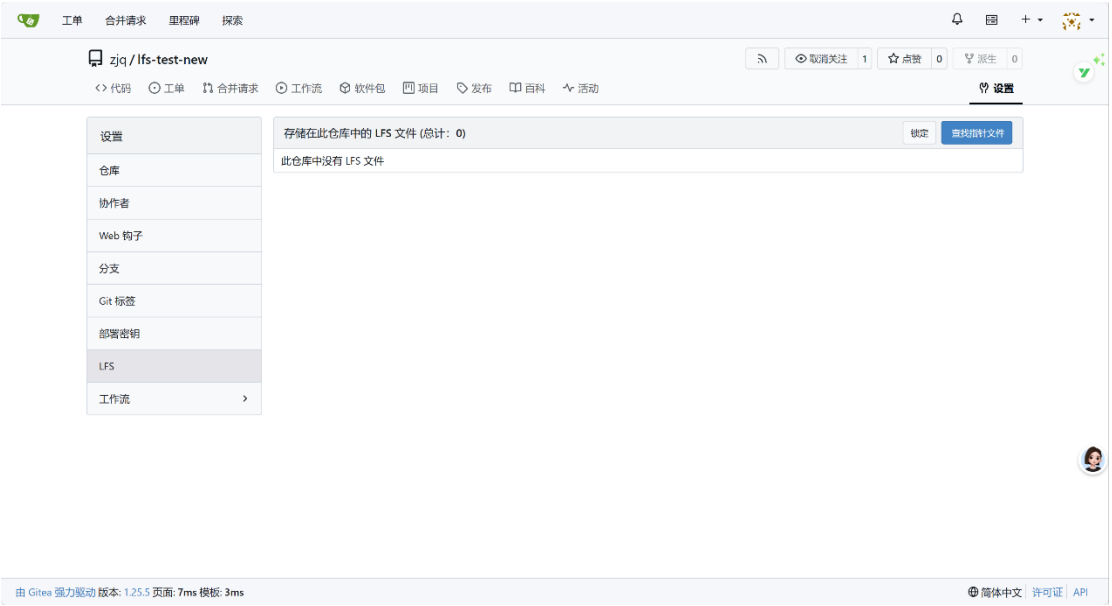
3.5 Step 5: Resolve "Non-fast-forward" Push Rejection and Force Push

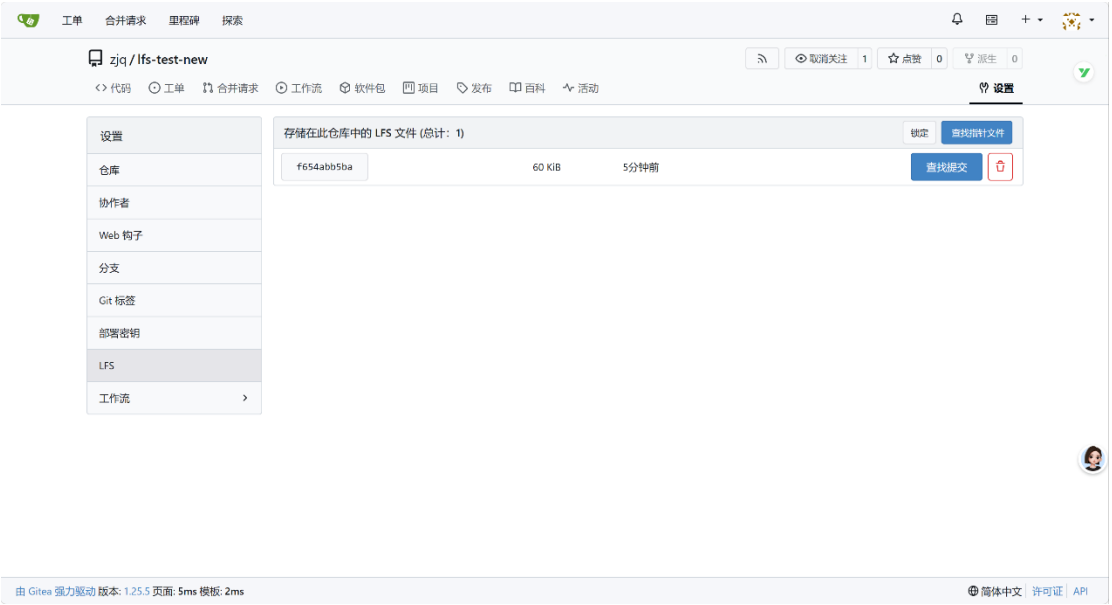
When pushing to the remote repository, Git rejects the push with `non-fast-forward` because the remote repository has initial files (e.g., README) that the local repository does not have. I resolve this by pulling remote changes and force-pushing the local branch.

```
zgg@LAPTOP-L21QMK1J:/mnt/d/Gitea-LFS-Test$ git push -u origin main --force
Username for 'http://localhost:3000': zjq
Password for 'http://zjq@localhost:3000':
Username for 'http://localhost:3000': zjq
Password for 'http://zjq@localhost:3000':
Locking support detected on remote "origin". Consider enabling it with:
  $ git config lfs.http://localhost:3000/zjq/lfs-test-new.git/info/lfs.locksverify true
Uploading LFS objects: 100% (1/1), 61 KB | 0 B/s, done.
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 22 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (7/7), 649 bytes | 15.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: . Processing 1 references
remote: Processed 1 references in total
To http://localhost:3000/zjq/lfs-test-new.git
 + e473d6d...2819526 main -> main (forced update)
branch 'main' set up to track 'origin/main'.
```

3.6 Step 6: Verify Git LFS Function on Gitea Web Interface

After successful push, access the Gitea web interface to confirm that the LFS file is stored normally.





4. Detailed Problem Description and Resolution

4.1 Problem 1: Git Identity Configuration Error

1. Symptom: When executing `git commit -m "init lfs"`, the terminal reports `fatal: empty ident name (for <zgg@LAPTOP-L21QMK1J.localdomain>) not allowed`.
2. Root Cause: Git requires a global user name and email to identify the commit author;

these were not configured initially.

3. Resolution: Run `git config --global user.name "zjq"` and `git config --global user.email "1794671582@qq.com"` to set global user information.

4.2 Problem 2: Untracked Files (LFS Tracking Not Effective)

1. Symptom: After executing `git add .` and `git commit`, the terminal prompts `Untracked files: liora.zip`, and no changes are committed.
2. Root Cause: The file `liora.zip` is not explicitly added to the LFS tracking list; `git add .` fails to include it due to incomplete LFS configuration.
3. Resolution: Explicitly execute `git add liora.zip` and `git add .gitattributes` to track the target file and configuration file.

4.3 Problem 3: Non-fast-forward Push Rejection

1. Symptom: When executing `git push origin main`, the terminal reports `! [rejected] main -> main (non-fast-forward)`, indicating that the remote branch has updated content that the local branch does not have.
2. Root Cause: The remote repository (created via Gitea web) has initial files (e.g., README), while the local repository is a new initialization with no related history. Git prohibits direct push to avoid overwriting remote history.
3. Resolution:

Pull remote unrelated history: `git pull origin main --allow-unrelated-histories`.

Force-push the local branch to the remote: `git push -u origin main --force` (applicable to experimental environments, overwrites remote history with local content).

5. Experimental Result Verification

5.1 Core Verification Result

From Gitea LFS settings page, we can see:

a) The LFS file count changes from `0` to `1`, proving that LFS is enabled and effective.

b) The LFS file size is displayed as `60 KiB`, confirming that the file is stored via Git

LFS (the file size does not affect the verification of LFS functionality; the core is that the LFS workflow is normal).

5.2 Operation Process Verification

From final push success log:

a) `Uploading LFS objects: 100% (1/1), 61 KB` confirms that the LFS object is successfully uploaded.
b) `branch 'main' set up to track 'origin/main'` confirms that the local branch is correctly associated with the remote branch.
c) `forced update` confirms that the local content is successfully synchronized to the remote Gitea repository.

6. Summary

In this experiment, I successfully deployed Gitea using Docker on WSL, resolved multiple common errors in Git LFS configuration and push operations, and verified that Gitea supports Git LFS storage. The key takeaways are:

1. Docker can quickly deploy Gitea with data persistence and correct port mapping.
2. Git LFS requires correct configuration of tracking types and user identity; explicit file tracking is required to avoid untracked file errors.
3. For independent local repositories and remote repositories with initial files, use `--allow-unrelated-histories` to pull and `--force` to push (experimental environment only) to resolve branch mismatch issues.
4. The core of the experiment is to verify the availability of Git LFS, and the size of the test file does not affect the validity of the experimental results.